

Danmarks
Tekniske
Universitet



Autonomous Driving Scenario

Perception for Autonomous Systems

AUTHORS

Anders Jensen - s194052
Lukas Egkher - s243430
Jóel Ingason - s242705
Teitur Tannason - 242739
Bozhi Lyu - s232251

December 5, 2024

Contents

1	Introduction	1
1.1	Provided data	1
1.2	General approach	1
2	Methods	2
2.1	Calibration and Rectification	2
2.1.1	Camera Calibration	2
2.1.2	Stereo camera calibration and image rectification	3
2.2	Depth Estimation	4
2.2.1	Disparity Map Computation	4
2.2.2	Depth Map Calculation	4
2.2.3	Post-Processing	4
2.2.4	Hyperparameter Tuning	4
2.3	Object Detection	5
2.4	3D Tracking	5
2.5	Evaluation	6
3	Results	7
3.1	Calibration and rectification results	7
3.2	Depth Estimation results	8
3.3	Object detection and segmentation results	9
3.3.1	Training and validation results	9
3.3.2	Model performance on sequences 1 and 2	11
3.4	Object tracking results	12
4	Conclusion	13
	References	I
	Appendix	II

1 Introduction

This project focuses on developing perception systems for autonomous driving in dynamic urban environments. The primary challenge is to detect, track, and classify pedestrians, cyclists, and cars in 3D, even under occlusion. A forward-facing stereo camera fixed on the vehicle provides visual data, requiring precise calibration and rectification to enable accurate object detection and classification as well as 3D-tracking. All the code for project can be found at GitHub and the video on Youtube.

1.1 Provided data

The project utilizes a diverse and comprehensive set of data, including three distinct sequences of stereo image pairs and supporting calibration information. Sequences one and two include rectified and raw stereo pairs with ground truth annotations, such as bounding boxes and depth information. Sequence three, on the other hand, also contains rectified and raw stereo images but lacks ground truth annotations. This absence of annotations makes it particularly valuable for testing.

Additionally, the dataset includes the calibration parameters of the stereo camera system, ensuring accurate depth estimation and 3D reconstruction. In a real-world scenario, only raw stereo data is typically available, requiring rectification to approximate true rectified images. To demonstrate the ability to rectify raw data and produce results as close as possible to true rectified images, the raw data is included. Rectified images are only provided to streamline subsequent steps. Together, this data provides a robust foundation for rectifying the raw data, detect and classify objects of interest and track them in 3D.

1.2 General approach

The project was divided into three main tasks, each logically building on the results of the previous step. The first task focused on rectifying the raw stereo images. This was achieved using different approaches described in Section 2, for the detection of chessboards of three different sizes. Using these detections, the left and right cameras were calibrated individually, followed by a stereo calibration. Based on these results, the raw images were rectified to obtain geometrically corrected stereo pairs.

The second task involved object detection using the YOLOv8 ultralytics model to identify and classify the desired objects. This step utilized the rectified stereo images to ensure accurate detection. Due to the extensive capabilities of YOLOv8, it was fine-tuned and restricted to detect only the relevant classes. This optimization ensured that the model was specifically tailored to the project's requirements while maintaining efficiency and accuracy.

In the third step, the focus shifted to tracking detected objects in 3D within a short video sequence. Using the calibrated and rectified stereo images, a depth map was constructed. Using the depth information along with the detection results gave us the 3D position of the detected object. We then used a modified version of ByteTrack [1] that added depth as a tracking parameter. Later on we also introduced a Re-identification model for feature matching and put spatial constraints on the matching of tracked objects.

2 Methods

This section outlines the methods employed to achieve reliable object detection and 3D tracking in urban environments, as well as the evaluation of the system's performance. The approach is divided into four key stages: calibration and rectification, object detection, 3D tracking, and evaluation.

2.1 Calibration and Rectification

The calibration process involved detecting multiple chessboards to estimate the camera matrices and then rectify the images. Each image contained 13 chessboards of 3 different sizes, which posed a challenge for automatic detection using OpenCV's `cv2.findChessboardCorners()`, as it can only detect a single chessboard per image.

2.1.1 Camera Calibration

Multiple approaches were tested, starting with a sliding window method. The image was divided into smaller sections with a specified size and overlap, and each smaller image was processed with the chessboard detection function. However, there are few limitations to this approach. Firstly, the width of windows and the step length affect the detection of chessboards. If a chessboard cannot be entirely included by any window, the detection will fail. In fact, only 7 out of 10 chessboards with 7×5 corners were detected by this method. Secondly, a single chessboard could be detected multiple times in different windows. However, because the calculated corners are `numpy.ndarray` of floating point numbers, it's hard to compare and filter out repeating elements in corners list, which results in a heavy computation.

The second approach involved manual region selection, where regions of interest were selected for each camera and treated as individual images. The detection rate for this method was very high, but it was time-consuming, inflexible and not automated at all.

The third approach automated the detection process by using a masking technique. Once a chessboard was detected, it was stored and then masked with a black box. The mask was transferred to the next image to ensure that a new chessboard would be detected. This approach checked three different pattern sizes to ensure that all chessboards were detected. The pattern sizes were 7×5 , 11×7 and 5×15 . Additionally, an alternative version of the detection function, `cv2.findChessboardCornersSB()`, was used. This function is more robust in highly distorted images, works better with larger images, and has built-in sub-pixel accuracy. This significantly improved detection performance and made it possible to detect 12 out of 13 chessboards. In order to improve the calibration the 13th chessboard was added by using a cropped version of the image, in which case it was detected. At this point we want to mention that we have no idea, why it worked for the cropped image but not for the original, already masked image, since we used the same self-written function. The result can be seen in figure 1.

The detected 3D object points and 2D image points of the corners from each camera were used to calculate the intrinsic camera parameters and distortion vectors. The calibration

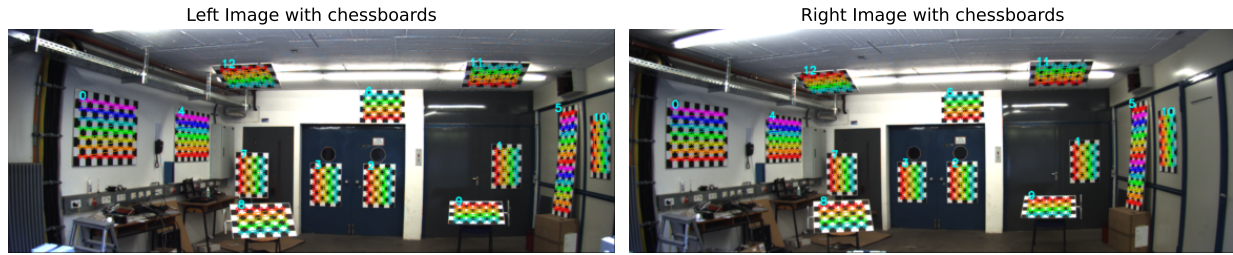


Figure 1: Detected chessboards with the labels corresponding to the iteration in which they were found.

accuracy of the left and right camera are validated as follows. First of all, we had a look on the undistorted image to get an expression of what we are dealing with - we don't include the figure here due to space limitations. Secondly, we tried to get a low mean reprojection error by trying different combination of flags for the `cv2.calibrateCamera()` function. Afterwards we compared the intrinsic camera matrices and the distortion vectors with the provided ones.

2.1.2 Stereo camera calibration and image rectification

Using the camera matrices and distortion vectors with `cv2.stereoCalibrate()`, the rotation matrix and translation were calculated. The chosen flags for the `stereoCalibrate` function had a large impact on the re-projection error and the whole stereo calibration. A script was made to automate the process of finding a reasonable combination of flags for stereo camera calibration by trying all the possible combinations of flags and check if the re-projection error was below a specified threshold. The `cv2.CALIB_FIX_ASPECT_RATIO` was performing best with a re-projection score below 0.13. This flag ensures that the aspect ratio of the focal lengths f_x, f_y remains fixed during the optimization process.

In order to get the desired rectification (rotation) and projection matrices we used the `cv2.stereoRectify()` function with the flag `cv2.CALIB_ZERO_DISPARITY`. This flag enforces the rectification transformation such that the epipolar lines in the rectified images are aligned and the disparity between the two images is minimized to zero under ideal conditions. Nevertheless, this flag may bring a small disadvantage by reducing the field of view, especially if the cameras were not perfectly aligned during capture. However, we didn't observe this negative side-effect. We also tried to set a new image size in the `cv2.stereoRectify()` function which we hoped would ensure that the left and right pictures have nearly the same size after the rectification. Furthermore, we calculated the undistortion and rectification transformation maps for both cameras with the use of the `cv2.initUndistortRectifyMap()` function and remapped our original images by applying this maps by using `cv2.remap()`. This adjusts the undistorted images using the projection and translation matrices returned from the rectification function.

Finally, a SIFT detector was implemented in order to visualize epipolar lines and confirm the rectification. We used the `cv2.BFMatcher` to match the 200 best matches.

2.2 Depth Estimation

The depth estimation process combines stereo disparity computation, depth map and post-processing to provide depth measurements. It utilizes stereo rectified image pairs and calibration parameters to produce depth maps for the object tracking.

2.2.1 Disparity Map Computation

The disparity maps are computed using OpenCV's StereoSBGM function[2], which is their Semi-Global Block Matching method. The disparity between corresponding points in the left and right rectified images is determined, providing a basis for calculating depth. The key parameters for the StereoSBGM.create function are: The starting disparity, number of disparities, block size, smoothness penalties. But it also includes additional parameters such as uniqueness ration, specklewindow size and speckle range. They are used for improving the accuracy by reducing noise and ensuring smoothness.

The grayscale input images are preprocessed using OpenCV's equalizeHist function to enhance contrast and improve the matching performance. The computed disparity maps are then normalized and refined using median filtering to reduce the noise.

2.2.2 Depth Map Calculation

The depth map is derived from the disparity map using the stereo camera's intrinsic and extrinsic calibration parameters. The depth Z is calculated, by dividing the focal length from the calibration matrix multiplied with the distance between the two camera centers, by the disparity. Invalid or extreme values, such as disparities less or equal than zero are masked out and we constrain the depth values to a range of 2 to 60 meters.

2.2.3 Post-Processing

The depth maps undergo additional refinement using both OpenCV's bilateralFilter[3], which smooths depth values while preserving edges and their inpaint[4] function which addresses invalid depth values using inpainting. This post-processing improves the visual quality and accuracy of the depth maps, especially in regions with low texture or occlusions.

2.2.4 Hyperparameter Tuning

To optimize the depth estimation pipeline, hyperparameter tuning was conducted using Optuna[5], and open-source optimization framework. It was used to optimize: numDisparities, blockSize, minDisparity, Smoothness penalties P1 and P2, Noise reduction parameters (speckleWindowSize, speckleRange, and uniquenessRatio). The objective function minimized the mean depth error by evaluating depth maps against ground truth labels. The dataset was split into training and validation subsets to ensure generalizability. The best parameter set was saved for future use, significantly enhancing the accuracy and robustness of the depth estimation process.

2.3 Object Detection

To detect objects in the pictures, we started with a pretrained YOLOv8 model [6]. Since the pre-trained model does not identify cyclists, only pedestrians or bicycles separately, we needed to train the model in order to identify and segment cyclists. Two public datasets [7] [8] were used to train the model. We used the model weights of `yolov8m.pt` as a starting point and trained for 50 epochs.

2.4 3D Tracking

The 3D tracking of objects was achieved by integrating stereo depth estimation with the ByteTrack code from [6] used as a reference and then improved, ensuring consistent tracking across frames even in dynamic and occluded environments. The tracking process relied on the detections from YOLOv8 and stereo matching techniques to calculate 3D positions of objects and maintain their identities over time.

Tracking with ByteTrack

To ensure temporal consistency of object identities across frames, the ByteTrack multi-object tracker was used. ByteTrack offers an approach to multi-object tracking (MOT) which addresses some limitations of traditional methods, which often discard low-confidence detections. Instead of relying solely on high-confidence detections, ByteTrack incorporates all detection boxes, leveraging their similarities with existing tracklets to recover true objects and filter out background noise. This strategy reduces object fragmentation and improves the accuracy of tracking in challenging conditions, such as occlusion.

As new detections were processed, their bounding boxes and depth estimations were provided to ByteTrack, which updated the state of each track. Each tracked object was represented by a unique identifier, a bounding box, a class label, and its depth estimation. This representation facilitated continuous monitoring of object motion and positioning in the 3D scene.

Improvements

To improve upon the tracking, and to make it work when objects were occluded for a long period of time, several adjustments were made, and some new features added. First a Re-identification feature was added. For each detection features were extracted from the bounding box, and added to the tracker. For the feature matching a pretrained resnet50l[9] model was used with the final classification layer removed. The features are the output of the last layer in the pretrained resnet50 model. Then when a new detection is made, the cosine distances between the saved feature and the new detection was calculated and used as a cost when matching tracks together.

A velocity constraint was also made to reduce false matches, so matches more than 200 pixels apart were considered invalid.

Hyperparameter tuning

To optimize the tracking pipeline, hyperparameter tuning was conducted using Optuna, an open-source optimization framework. The tuning process focused on optimizing key parameters for the ByteTrack algorithm, including detection-to-track matching thresholds such as `match_thresh` and `new_track_thresh`, buffer size and depth related thresholds such as `track_buffer`, `track_high_thresh` and `track_low_thresh`. We tuned specific parameter sets, for different object classes, pedestrians, cars and cyclists, to account for their unique motion patterns and appearance characteristics.

The objective function aimed to minimize the negative Multiple Object Tracking Accuracy (MOTA), factoring in precision and penalties for false matches and extra tracks. Tracking performance was evaluated against ground truth labels, with the dataset split into training and validation subsets to ensure generalizability. The best hyperparameter configurations for each object class were saved and integrated into the pipeline.

2.5 Evaluation

To assess the performance of the object detection and 3D tracking system, the labeled ground truth data from sequences 1 and 2 was used for evaluation. The evaluation focused on measuring two primary metrics: precision and recall, which are widely used to quantify the accuracy and reliability of object detection and tracking systems.

The evaluation process involved comparing the tracked objects with the corresponding ground truth labels for each frame in the sequences. Precision was calculated as the ratio of correctly identified objects to the total number of predicted objects, indicating the system's ability to avoid false positives. Recall, on the other hand, measured the ratio of correctly identified objects to the total number of ground truth objects, capturing the system's capability to detect relevant objects.

The evaluation was implemented using a custom Evaluator class, which computed these metrics for each frame and aggregated the results to report the overall performance. During this process, tracked objects were matched with ground truth labels based on their class and proximity, ensuring a fair and accurate comparison. Precision and recall values were logged for each frame, and the average metrics were computed across the entire dataset to provide a comprehensive performance summary.

The results demonstrated the system's ability to detect and track objects reliably in challenging urban scenarios, providing a solid foundation for further analysis and development. Sequence 3, which lacked ground truth annotations, was used for qualitative evaluation and testing under real-world conditions, highlighting the robustness of the system.

3 Results

3.1 Calibration and rectification results

The calibration accuracy of the left and right camera are validated in many ways as described in section 2. We minimized the reprojection errors with 0.13 for the left camera calibration and 0.14 for the right one. The corresponding intrinsics camera matrices and distortion vectors were not that far off from the provided correct ones in the `calib_cam_to_cam.txt` file. Except from the k_2 distortion parameter which we couldn't reach at all. All numbers can be found in tables 1, 2.

The rectification turned out to be really good, which can be seen when comparing the correctly rectified image and our calculated rectified one in figure 2. We got slightly more pixels than the true rectified pictures but it seems that we therefore, have a little more information in the picture.

We also adjusted the value of alpha in the `cv2.stereoRectify()` function and came to the conclusion that $\alpha = 0.7$ together with the region of interest computed the best results. Also, the projection and rectification matrices look good in this case. Just the last column of the projection matrices (e.g. the translation) is not very good at all. Especially for the x coordinate there is a huge difference. The values can be found in tables 3, 4. In general, we came pretty close to the underlying true data.

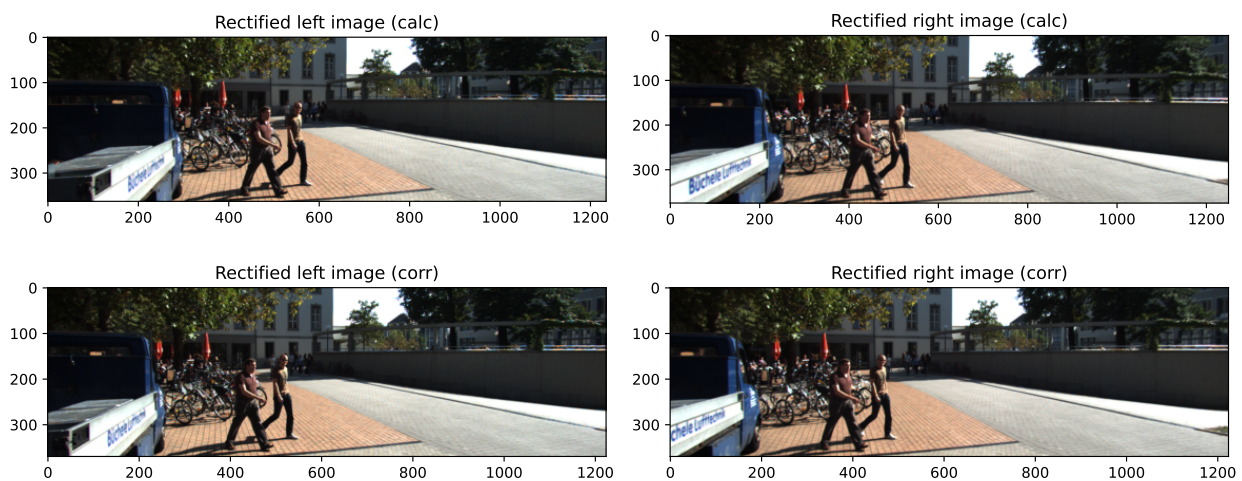


Figure 2: Comparison between our calculated rectification and the true one visualised on the first provided image from sequence 1

Another way of proofing the rectification worked out is to plot the epipolar lines. Since they are perfectly horizontal, we conclude, that the rectification worked. A plot of the 200 best keypoints and the epipolar lines can be seen in figure 3. This also helped us to understand, that simply resetting the image size in the `cv2.stereoRectify()` function will mostly harm the rectification as our epipolar lines were not even close to horizontal when using this optional argument.

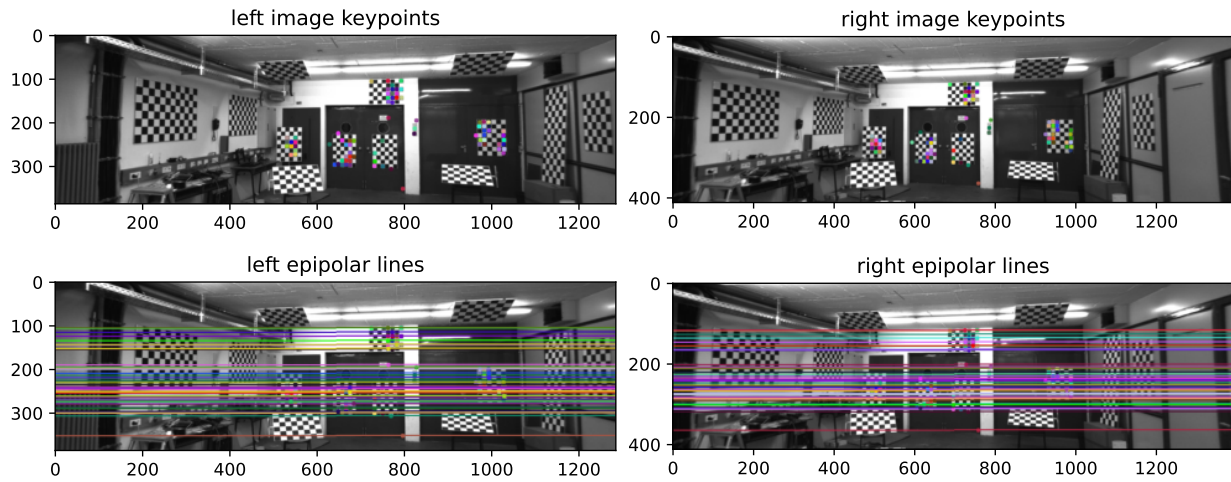
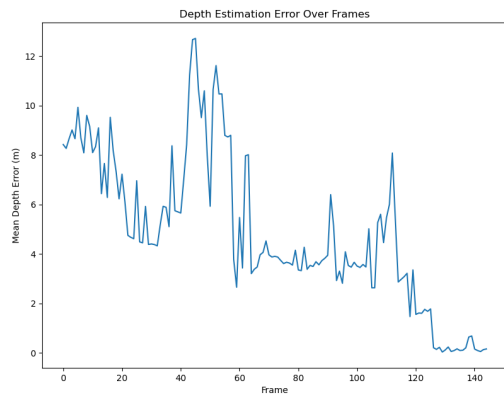


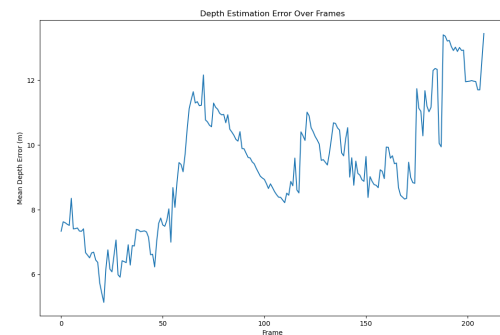
Figure 3: Epipolar lines based on 200 best keypoints in image 0000000011.png

3.2 Depth Estimation results

The depth estimation performance was evaluated using mean depth error metrics across frames for sequences 1 and 2. The results are illustrated in Figures 4a and 4b.



(a) Mean depth error for sequence 1



(b) Mean depth error for sequence 2

Figure 4: Mean depth error over frames for sequences 1 and 2.

The results show that the depth estimation model performs reasonably well, with a consistent decrease in error across many frames in sequence 1. However, sequence 2 exhibits higher fluctuations in mean depth error, particularly in later frames. These fluctuations are probably due to challenges with mike, because of the occlusions he causes in sequence 2.

Despite these challenges, the depth estimation performance is reasonable and aligns with expectations for stereo-based depth estimation methods.

3.3 Object detection and segmentation results

3.3.1 Training and validation results

After 50 epochs the metrics stopped improving and we deemed the accuracy of the model acceptable. The training and validation results can be seen in figure 5.

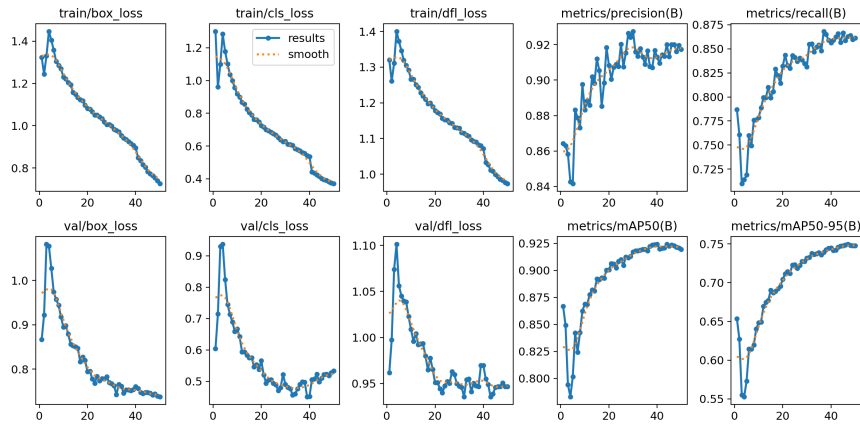


Figure 5: Training results

In figure 6 the results of one validation batch can be seen with the predicted data. In figure 7 we can compare this to the ground truth.



Figure 6: Model predictions

Figure 8 shows various parameters used to evaluate the performance of the model. As seen in the confusion matrix, the model struggles with overidentifying both pedestrians and cars when only the background is present. This could possibly be mitigated with a larger dataset. We can also see that it sometimes detects objects when no objects should be detected.

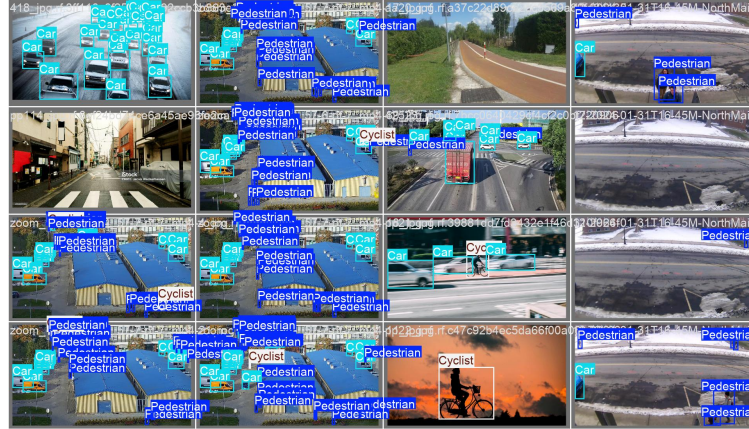


Figure 7: Ground truth

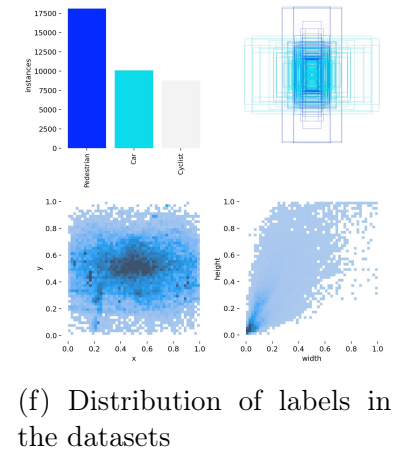
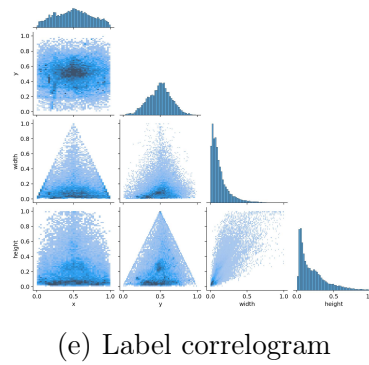
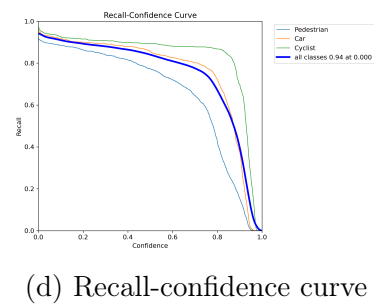
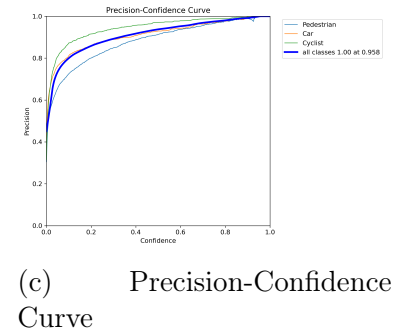
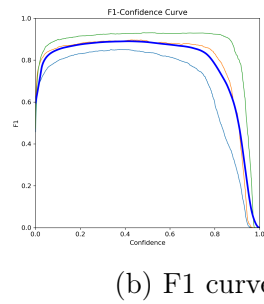
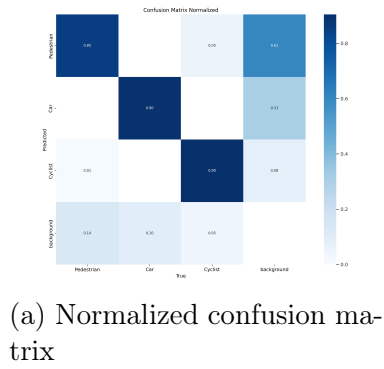
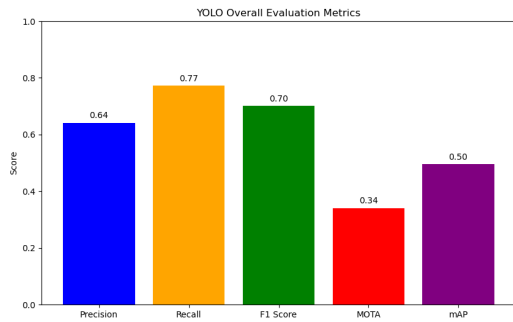


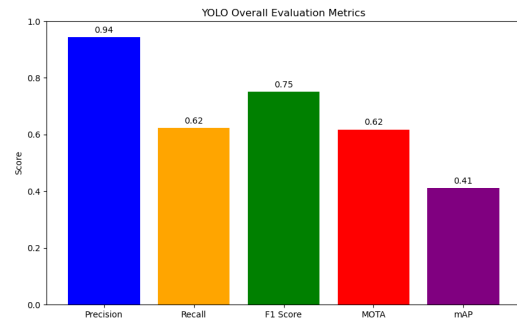
Figure 8: Various parameters to estimate model training performance and quality of dataset

3.3.2 Model performance on sequences 1 and 2

The trained model did perform well on sequences 1 and 2 but compared to the ground truth it did not always identify everything in the image and sometimes mislabeled objects. The model did identify a car in sequence 1 that the ground truth labels do not include, but it is arguably a car. As can be seen in figure 9 and 10 the ground truth labels do not always include every object in the image and sometimes seem to not have perfect bounding boxes.



(a) Overall validation metrics for sequence 1.

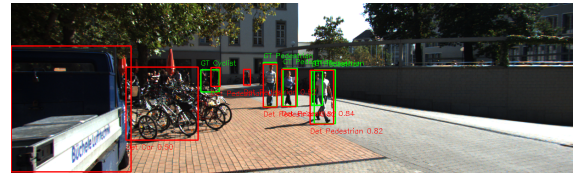


(b) Overall validation metrics for sequence 2.

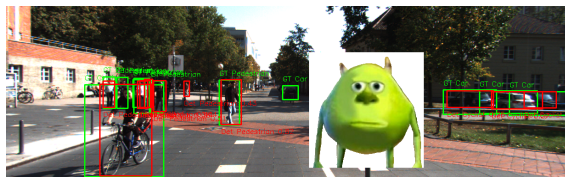
Figure 9: Overall validation results when running the detection model on sequence 1 and 2.



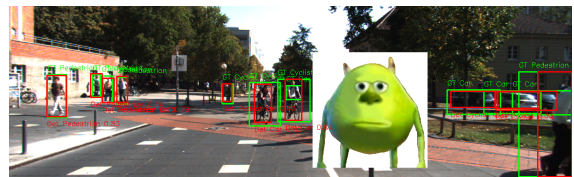
(a) Example of ground truth vs model detection in sequence 1.



(b) Example of ground truth vs model detection in sequence 1.



(c) Example of ground truth vs model detection in sequence 2.

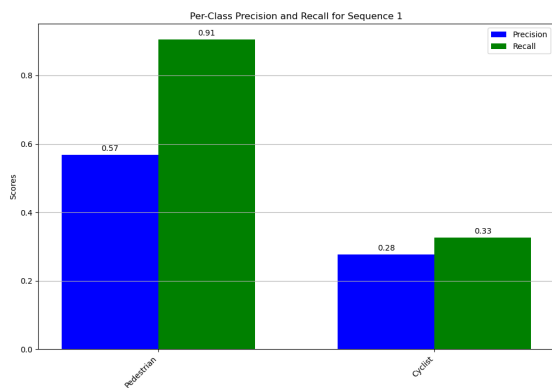


(d) Example of ground truth vs model detection in sequence 2.

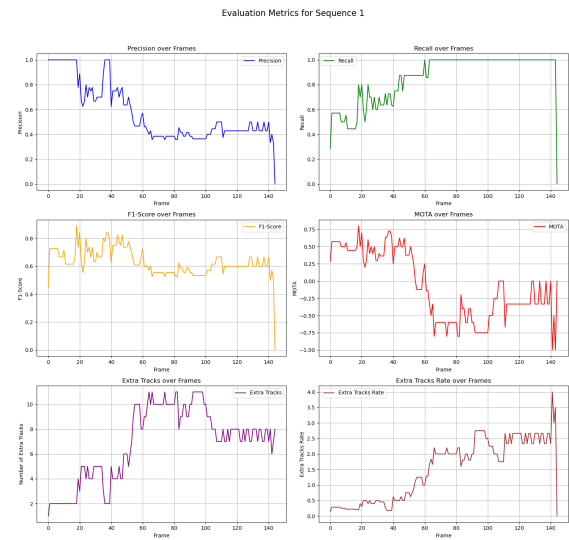
Figure 10: Examples from running the detection model on sequences 1 and 2. The red boxes are predictions from the model and the green boxes are the ground truth.

3.4 Object tracking results

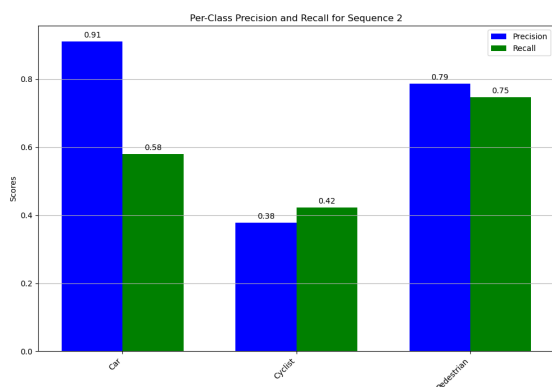
Overall, the object tracking went well but we did see that it fails to match tracks occasionally. The situation where the model most often failed was in sequence two, when a lot of pedestrians and cyclists are being tracked and occluded at the same time. The hyperparameter tuning could probably have been run longer in order to achieve better results. The evaluation metrics for sequence one and two can be seen in figure 11. The tracking seems to struggle the most on cyclists and sometimes on pedestrians as well.



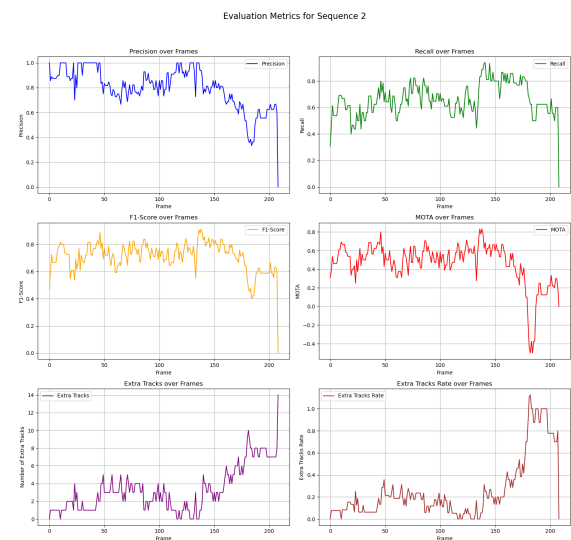
(a) Per class precision and recall for sequence 1.



(b) Evaluation metrics for sequence 1.



(c) Per class precision and recall for sequence 2.



(d) Evaluation metrics for sequence 2.

Figure 11: Evaluation of tracking performance on sequences 1 and 2.

4 Conclusion

The rectification of the raw images turned out to be a lot more difficult than expected. The key part was to find the best flags to get the most out of the algorithms from the OpenCV library. What helped a lot was choosing the flags based on the returned re-projection error from OpenCV's calibration functions. Also, drawing the epipolar lines was an effective way to validate the rectification.

For the object detection we chose a strong model with the YOLOv8 where the biggest challenge was fine-tuning the model with two other datasets to adapt it to our requirements.

In order to track the object in 3D we had to estimate the depth. We had a lot of struggles with the mean depth error, as it raised over the frames in sequence 2. Additionally, there were some issues, when many objects crossed their paths where we lost track of some for a few frames. But this was only a rare phenomenon and if so, the object got detected again just afterwards. We come to the conclusion that these are the difficulties of working with real-life data and occluded images. There is definitely room for improvement. Apart from that, the tracking performed good.

To sum it up, this project gave us a better understanding on which challenges might pop up, when working on a somewhat realistic project and improved our knowledge on how to deal with them.

References

- [1] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “Bytetrack: Multi-object tracking by associating every detection box,” 2022.
- [2] G. Bradski, “StereoSGBM Class - OpenCV Documentation.” https://docs.opencv.org/4.x/d2/d85/classcv_1_1StereoSGBM.html. Accessed: 2024-12-03.
- [3] G. Bradski, “Bilateral Filter Function - OpenCV Documentation.” https://docs.opencv.org/4.x/d4/d86/group_imgproc_filter.html#ga9d7064d478c95d60003cf839430737ed. Accessed: 2024-12-03.
- [4] G. Bradski, “Inpainting Tutorial - OpenCV Documentation.” https://docs.opencv.org/3.4/df/d3d/tutorial_py_inpainting.html. Accessed: 2024-12-03.
- [5] G. Bradski, “Optuna - An Optimization Framework.” <https://optuna.org/>. Accessed: 2024-12-03.
- [6] G. Jocher, J. Qiu, and A. Chaurasia, “Ultralytics YOLO,” Jan. 2023.
- [7] S. Bernabe, “Smart city pedestrian/bike detection dataset.” <https://universe.roboflow.com/simone-bernabe/smart-city-pedestrian-bike-detection>, oct 2024. visited on 2024-12-03.
- [8] cjff, “Smart city pedestrian/bike detection dataset.” <https://universe.roboflow.com/cjff/smart-city-pedestrian-bike-detection-s4skz>, aug 2024. visited on 2024-12-03.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

Appendix

	$\mathbf{f}_x$	$\mathbf{f}_y$	$\mathbf{c}_x$	$\mathbf{c}_y$
$\mathbf{K}_{\text{left}}^{\text{cal}}$	972.49	972.49	689.54	255.23
$\mathbf{K}_{\text{left}}^{\text{corr}}$	956.95	952.24	693.98	238.61
$\mathbf{K}_{\text{right}}^{\text{cal}}$	895.42	901.75	687.05	255.54
$\mathbf{K}_{\text{right}}^{\text{corr}}$	901.10	897.06	698.29	237.74

Table 1: Intrinsic camera coefficients

	k_1	k_2	p_1	p_2	k_3
$\mathbf{D}_{\text{left}}^{\text{cal}}$	-0.27	-0.00	0.00	-0.00	0.06
$\mathbf{D}_{\text{left}}^{\text{corr}}$	-0.38	0.21	0.00	0.00	-0.07
$\mathbf{D}_{\text{right}}^{\text{cal}}$	-0.28	0.00	-0.00	-0.00	0.07
$\mathbf{D}_{\text{right}}^{\text{corr}}$	-0.37	0.19	-0.00	0.00	-0.06

Table 2: Distortion coefficients

$$\begin{aligned}
\mathbf{R}_{\text{left}}^{\text{cal}} &= \begin{bmatrix} 1.00 & 0.01 & -0.00 \\ -0.01 & 1.00 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{bmatrix} & \mathbf{R}_{\text{right}}^{\text{cal}} &= \begin{bmatrix} 1.00 & -0.01 & 0.01 \\ 0.01 & 1.00 & 0.00 \\ -0.01 & 0.00 & 1.00 \end{bmatrix} \\
\mathbf{R}_{\text{left}}^{\text{corr}} &= \begin{bmatrix} 1.00 & 0.02 & -0.01 \\ -0.02 & 1.00 & -0.00 \\ 0.01 & 0.00 & 1.00 \end{bmatrix} & \mathbf{R}_{\text{right}}^{\text{corr}} &= \begin{bmatrix} 1.00 & -0.01 & 0.02 \\ 0.01 & 1.00 & -0.00 \\ -0.02 & 0.00 & 1.00 \end{bmatrix}
\end{aligned}$$

Table 3: Rectification matrices

	f_x	f_y	c_x	c_y	t_x	t_y	t_z
$\mathbf{P}_{\text{left}}^{\text{cal}}$	702.12	702.12	640.50	199.81	0.00	0.00	0.00
$\mathbf{P}_{\text{left}}^{\text{corr}}$	707.05	707.05	604.08	180.51	45.76	-0.35	0.00
$\mathbf{P}_{\text{right}}^{\text{cal}}$	702.12	702.12	640.50	199.81	-3753.65	0.00	0.00
$\mathbf{P}_{\text{right}}^{\text{corr}}$	707.05	707.05	604.08	180.51	-334.11	2.33	0.00

Table 4: Projection coefficients